

```
In [1]: def read_file(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            print(content)
    except FileNotFoundError:
        print(f"File '{filename}' not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

# Usage:
read_file("my_file.txt") # Replace with your actual file name
```

File 'my_file.txt' not found.

```
In [2]: def copy_file(source_filename, target_filename):
    try:
        with open(source_filename, 'r') as source_file:
            content = source_file.read()
            with open(target_filename, 'w') as target_file:
                target_file.write(content)
        print(f"Contents copied from '{source_filename}' to '{target_filename}'.")
    except FileNotFoundError:
        print(f"File '{source_filename}' not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

# Usage:
copy_file("source.txt", "target.txt") # Replace with actual file names
```

File 'source.txt' not found.

```
In [3]: def count_words(filename):
    try:
        with open(filename, 'r') as file:
            content = file.read()
            word_list = content.split()
            word_count = len(word_list)
            print(f"Total words in '{filename}': {word_count}")
    except FileNotFoundError:
        print(f"File '{filename}' not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

# Usage:
count_words("sample.txt") # Replace with your actual file name
```

File 'sample.txt' not found.

```
In [4]: def get_integer_input():
    try:
        user_input = input("Enter an integer: ")
        integer_value = int(user_input)
        print(f"Converted integer: {integer_value}")
    except ValueError:
        print("Invalid input. Please enter a valid integer.")

# Usage:
get_integer_input()
```

Invalid input. Please enter a valid integer.

```
In [5]: def check_positive_integers():
    try:
        user_input = input("Enter a list of integers (comma-separated): ")
        integer_list = [int(x) for x in user_input.split(',')]
        if any(num < 0 for num in integer_list):
            raise ValueError("Negative integers are not allowed.")
        print("All integers are non-negative.")
    except ValueError as ve:
        print(f"Error: {ve}")

# Usage:
check_positive_integers()
```

Error: Negative integers are not allowed.

```
In [6]: def check_positive_integers():
    try:
        user_input = input("Enter a list of integers (comma-separated): ")
        integer_list = [int(x) for x in user_input.split(',')]
        if any(num < 0 for num in integer_list):
            raise ValueError("Negative integers are not allowed.")
        print("All integers are non-negative.")
```

```
except ValueError as ve:
    print(f"Error: {ve}")
```

```
# Usage:
check_positive_integers()
```

Error: invalid literal for int() with base 10: '345678-9'

```
In [7]: def compute_average():
        try:
            user_input = input("Enter a list of integers (comma-separated): ")
            integer_list = [int(x) for x in user_input.split(',')]
            average = sum(integer_list) / len(integer_list)
            print(f"Average: {average:.2f}")
        except ValueError:
            print("Invalid input. Please enter valid integers.")
        finally:
            print("Program finished running.")

# Usage:
compute_average()
```

Invalid input. Please enter valid integers.
Program finished running.

```
In [8]: def compute_average():
        try:
            user_input = input("Enter a list of integers (comma-separated): ")
            integer_list = [int(x) for x in user_input.split(',')]
            average = sum(integer_list) / len(integer_list)
            print(f"Average: {average:.2f}")
        except ValueError:
            print("Invalid input. Please enter valid integers.")
        finally:
            print("Program finished running.")

# Usage:
compute_average()
```

Invalid input. Please enter valid integers.
Program finished running.

```
In [9]: def write_to_file(filename, content):
        try:
            with open(filename, 'w') as file:
                file.write(content)
            print(f"String written to '{filename}'.")
        except Exception as e:
            print(f"An error occurred: {e}")

# Usage:
user_string = input("Enter a string to write to a file: ")
write_to_file("output.txt", user_string) # Replace with your desired filename
```

String written to 'output.txt'.

In []: